PROPERTISM TECHNICAL REFERENCE & SECURITY GUIDE

🤖 How Google reCAPTCHA & DB Keys Defend Against Spam

Architectural Breakdown: Human Behavior vs. Automated Exploitation

Product: Propertism (03)
Date: 2026-10-18
Supervisor: Astra 1
Owner: Viji

> *"To a human, it looks like a simple checkbox. Under the hood, it is one of the most sophisticated AI and behavioral analysis systems on the web."*

🔍 1. The Illusion vs. The Reality

To a visitor, the process appears completely trivial:

> ***"I just clicked a box that says 'I'm not a robot' — how hard is it for a computer program to click a box?"***

The secret is: **the checkbox is just a distraction.** The actual test happens in the critical milliseconds **before, during, and after** the cursor moves toward that box.

🛠️ 2. The 3 Behavioral Defensive Layers Behind the Box

① The Physics of Mouse Movement (Human Chaos vs. Bot Rigidity)

- **Humans are chaotic & organic:** When a real person moves their hand across the screen, the trajectory curves naturally, decelerates upon approaching the target, exhibits micro-tremors, and pauses slightly before clicking.
- **Bots are mathematically rigid:** Scripted bots snap the cursor in mathematically perfect straight lines, or teleport instantly to coordinates with zero travel time, firing a synthetic `click()` event.
- Google's tracking script records the **precise trajectory, velocity, acceleration, and micro-movements** of your mouse across the viewport.

② Browser Fingerprinting & Environment Inspection

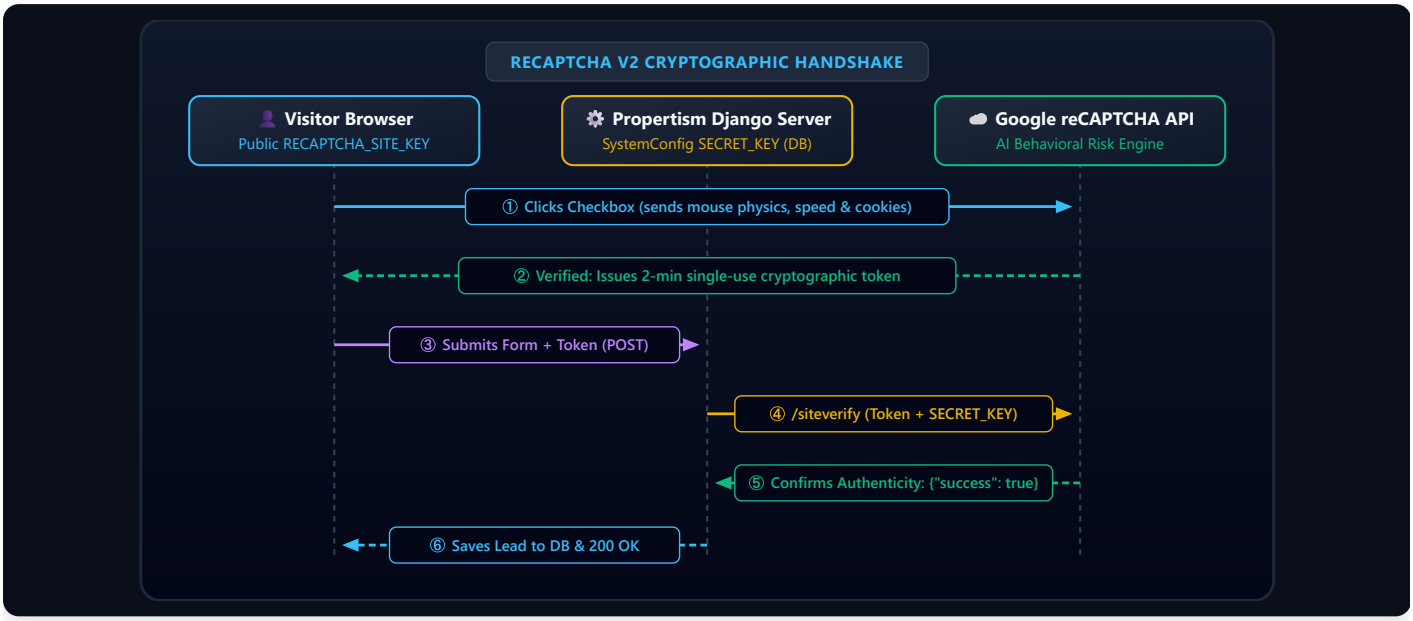
- **Time on Page:** Genuine visitors spend 10 to 30 seconds reading and typing Name, Phone, and Email. Spam bots submit within 150 milliseconds of socket opening.
- **Google Identity & Session Cookies:** If the visitor has an active Google session (Gmail, YouTube, Search), Google's risk engine recognizes the machine as an authenticated, trusted human profile.
- **Real Hardware vs. Headless Scripts:** Bots typically run headless runners (Puppeteer, Selenium, Chromium CLI). Google probes WebGL graphics drivers, audio hardware, and system fonts to verify real physical hardware.

③ The Puzzle Challenge (Fallback Visual Gate)

- If mouse movement physics and browser metrics pass, Google immediately issues the **green checkmark**.
- If behavioral metrics trigger suspicion (datacenter IP, erratic physics, or zero browsing history), Google denies automatic approval and forces the visual challenge: *"Select all squares with traffic lights, buses, or crosswalks."*

🔐 3. The Cryptographic Handshake (Why We Need DB Keys)

This sequence explains why both `RECAPTCHA_SITE_KEY` (public frontend) and `RECAPTCHA_SECRET_KEY` (private backend database) are mandatory:



⚙️ How the Verification Steps Work Under the Hood

- **1. Token Generation:** When the user passes the checkbox, Google issues a single-use cryptographic token (`g-recaptcha-response`) valid for exactly **2 minutes**.
- **2. Form Submission:** The browser attaches this one-time token into the hidden form field alongside the user's name, email, and phone number upon clicking “Schedule Free Consultation”.
- **3. Server-to-Server Verification:** Propertism’s Django server pauses lead creation and contacts Google’s private API: *“Hey Google, this user submitted token `03AFc...`. Here is our private `RECAPTCHA_SECRET_KEY` from PostgreSQL to verify identity. Is this submission legitimate?”*
- **4. Google Approves:** Google responds with `{"success": true}`. Only after this handshake succeeds does Django commit the lead to PostgreSQL and send email alerts.

🛡️ 4. Root Cause Analysis: How Yesterday's Vulnerabilities Were Sealed

❌ Previous Vulnerabilities (Why Bots Infiltrated)

- **Missing Secret Key Loophole:** If `RECAPTCHA_SECRET_KEY` was missing from the server environment, legacy code had a flaw that assumed `success = True` !
- **Public API Bypass:** Bots bypassed web browsers entirely, sending automated JSON directly to `/api/inquiries/` with zero captcha verification.
- **Fail-Open Setting:** If Google’s API had a network hiccup or timeout, `RECAPTCHA_FAIL_OPEN=True` let submissions slip through.

✅ Production Hardening Applied (Today's Shield)

- **DB-Backed SystemConfig:** Credentials reside directly in PostgreSQL table `content_systemconfig` , immune to missing server `.env` files.
- **Locked API Gate:** `/api/inquiries/` enforces `IsStaffOrDealEngineKey` (returns `HTTP 401 Unauthorized` to scripts).
- **Strict Fail-Closed:** Unverified, expired, or missing tokens are permanently rejected with `HTTP 400 Bad Request` .